TECHNICAL DOCUMENT · SEMICONDUCTOR TEST

AI-Optimized Fmax Search for Test Time Reduction

Principles, algorithms, data, quality controls and production deployment for machine-learning-assisted speed binning



Contents at a glance

- Conventional Fmax search and its cost model
- Taxonomy of AI-optimized approaches
- Features, models and calibrated uncertainty
- Four production algorithms with pseudocode
- Simulation study: linear, binary, Bayesian and ML-assisted search
- Quality, risk management and deployment architecture
- Published results and implementation roadmap

Version 1.0 · October 2026

Simulation results in Section 7 use synthetic data and are illustrative.

Contents

Executive summary . . . 3
1 Introduction . . . 3
1.1 Fmax and speed binning . . . 3
1.2 Why the search dominates test time . . . 4
1.3 Scope . . . 4
2 Conventional Fmax search . . . 4
2.1 Search strategies . . . 4
2.2 Error sources . . . 5
3 AI-optimized approaches . . . 5
3.1 A: Predictive binning (replace the search) . . . 5
3.2 B: Prediction-windowed search (narrow the search) . . . 6
3.3 C: Probabilistic (Bayesian) adaptive search . . . 6
3.4 D: Reuse across corners . . . 6
3.5 E: Pattern selection and ordering . . . 6
3.6 F: Label-efficient learning . . . 6
4 Data and features . . . 7
4.1 Feature sources . . . 7
4.2 Data pipeline requirements . . . 7
4.3 Label quality . . . 7
5 Modelling . . . 8
5.1 Model families . . . 8
5.2 Calibrated uncertainty . . . 8
5.3 Evaluation metrics . . . 8
6 Algorithms . . . 9
7 Simulation study . . . 10
7.1 Set-up . . . 10
7.2 Search cost . . . 11
7.3 Sensitivity to model accuracy . . . 12
7.4 Prediction-only binning and guard bands . . . 13
7.5 Limitations of the study . . . 13
8 Quality and risk management . . . 13
8.1 Failure modes . . . 13
8.2 Out-of-distribution fallback . . . 14
8.3 Audit sampling . . . 14
8.4 Drift monitoring . . . 14
8.5 Product categories with strict quality requirements . . . 14
9 Deployment architecture . . . 14
9.1 Integration with the tester . . . 15
9.2 Model lifecycle . . . 15
9.3 Estimating the benefit . . . 15
10 Reported results . . . 16
11 Implementation roadmap . . . 16
12 Conclusions . . . 17
References . . . 17
Appendix A Glossary . . . 18

Executive summary

Finding a device's maximum operating frequency (Fmax) is one of the most time-consuming steps in speed binning. A conventional flow applies a set of at-speed or functional tests at many clock frequencies until the pass/fail boundary is found, and repeats that search at each voltage and temperature corner that defines a bin. Because every unit goes through the same blind search, most of the tester time is spent confirming what could have been predicted from data already collected earlier in the flow.

AI-optimized Fmax search uses that earlier data, chiefly on-chip ring-oscillator and speed-monitor counts, leakage, Vmin and structural test results, to predict each unit's Fmax with a calibrated uncertainty. The tester then either confirms the predicted bin with one or two tests, or runs a short search only inside the predicted window. Published work reports reductions ranging from roughly half of the system-level search applications to three or four orders of magnitude when the functional search is removed entirely.

Key findings of this document

- **Windowing is the safe default.** In the simulation study (Section 7), a binary search restricted to the model's prediction interval needed 3.9 tests per part versus 6.5 for full binary search and 40.5 for a linear sweep: **40%** less search time than binary and **90%** less than linear, with the result still a measured Fmax.
- **Confirm-only binning goes further** (1.77 tests per part, 73% below binary search) when only the bin, not the exact Fmax, is needed.
- **Naive seeding is not enough.** Starting a linear walk at the predicted value barely beats binary search and becomes worse than it once model error exceeds about three grid steps. How the prediction is used matters as much as how accurate it is.
- **Prediction-only binning is risky.** With a model error of about 2.5% of Fmax and 100 MHz bins, keeping over-bin escapes below 100 ppm required a guard band of about 3 sigma, which pushed roughly three quarters of parts into a lower bin. Use pure prediction only where model error is small relative to bin width.
- **Production safety comes from the flow, not the model:** calibrated intervals, out-of-distribution fallback to a full search, a small audited sample with full search, and drift monitoring with retraining.

1 Introduction

1.1 Fmax and speed binning

Process variation means that dies from the same wafer, and even neighbouring dies, can run at noticeably different maximum frequencies. Speed binning sorts parts into frequency grades (or, for DVFS products, into voltage-frequency operating tables) so that faster silicon can be sold at a premium and slower silicon still ships at a lower grade. Determining each part's Fmax, or at least confirming which bin it belongs to, is therefore a production step with direct revenue impact.

Fmax can be measured in several ways: by applying functional patterns or a workload at increasing clock frequencies; by structural at-speed scan tests (launch-on-capture and launch-on-shift transition patterns applied with a fast capture clock); by system-level test (SLT) running real software; or indirectly from on-chip sensors such as ring oscillators and critical-path replicas. Functional and SLT measurements are the most trusted but also the slowest and most equipment-intensive.

1.2 Why the search dominates test time

For a single voltage and temperature corner, the time to find Fmax is approximately

$$T_{Fmax} = N_{steps} \times (t_{switch} + t_{apply})$$

where N_{steps} is the number of frequencies tried, t_{switch} is the time to change and settle the clock (PLL relock, power-supply settling, re-initialisation) and t_{apply} is the time to apply the pattern set or workload at that frequency. For a search window of width W and resolution Δf, a linear sweep needs on the order of $W/\Delta f$ steps, while a binary search needs

$$N_{binary} = \lceil log_2(W / \Delta f + 1) \rceil$$

The total cost multiplies further by the number of corners: a DVFS product binned at four voltage points and two temperatures needs eight searches. In SLT, where t_{apply} can be hundreds of milliseconds to seconds because the device must boot and run code, Fmax search can account for a large fraction of the total insertion time.

There are only two levers: reduce N_{steps} (search fewer frequencies) or reduce t_{apply} (spend less time at each frequency). AI-based methods act on both, but the largest and most reliable gains come from reducing N_{steps} using a per-unit prediction.

1.3 Scope

This document covers the principles, algorithms, data requirements, modelling choices, quality controls and deployment architecture for AI-optimized Fmax search in high-volume production. It includes a reproducible simulation study on synthetic data that compares conventional and AI-assisted search strategies under a common test-noise and binning model, and a summary of results reported in the literature. Numbers from the simulation are illustrative and should be re-established on the target product's own data before deployment.

2 Conventional Fmax search

2.1 Search strategies

Linear sweep

The clock is stepped upward from a safe starting frequency until the first failure; Fmax is the last passing frequency. It is simple, robust to occasional noisy results near the boundary, and gives a natural shmoo, but its cost grows linearly with the window and with how far the part is from the starting point.

Binary search

The window is halved at each step according to the pass/fail outcome. The number of steps is fixed and logarithmic in the window, which is why it is the common production default. Its weakness is sensitivity to test noise: a single wrong outcome early in the search sends it into the wrong half of the window, and the error is never corrected.

Coarse–fine and hybrid searches

A coarse step locates the boundary approximately and a fine step resolves it. Many test programs also start from a fixed, product-level guess rather than from the bottom of the window. These variants help on average but still treat every unit the same way.

Multi-corner (shmoo) searches

For DVFS binning the search is repeated at several supply voltages. Because Fmax rises smoothly with voltage, the result at one corner constrains the next, an observation that adaptive methods exploit (Section 3.4).

Table 1. Conventional Fmax search strategies.

Strategy	Steps per corner	Noise robustness	Typical use
Linear sweep	~W/Δf (tens)	Good; first fail stops the walk	Characterisation, shmoo plots
Binary search	$\lceil \log_2(W/\Delta f+1) \rceil$ (6–8)	Poor; early wrong outcome is not corrected	Production default
Coarse–fine	$W/\Delta f_c + \Delta f_c/\Delta f$	Moderate	Production with wide windows
Fixed-seed walk	depends on seed distance	Good	Mature products, narrow distributions

2.2 Error sources

- **Test repeatability.** Supply noise, temperature drift and pattern-dependent effects make the pass/fail boundary fuzzy over a few MHz; repeated tests at the same frequency do not always agree.
- **Quantisation.** Fmax is only resolved to the grid step Δf; reported values are biased low by about half a step.
- **Test-to-system correlation.** Structural tests can be optimistic or pessimistic relative to the functional or system Fmax, so a structural result is usually mapped to system speed by a correlation model.
- **Condition dependence.** Fmax depends on voltage, temperature and ageing margin; each must be fixed or guard-banded.

These errors are why any binning decision applies a measurement guard band: a part must pass some margin above the bin frequency to be placed in that bin. AI-optimized search must preserve, not weaken, these protections.

3 AI-optimized approaches

The approaches below differ in where the learned model acts. They are not exclusive, and a mature flow often combines several of them.

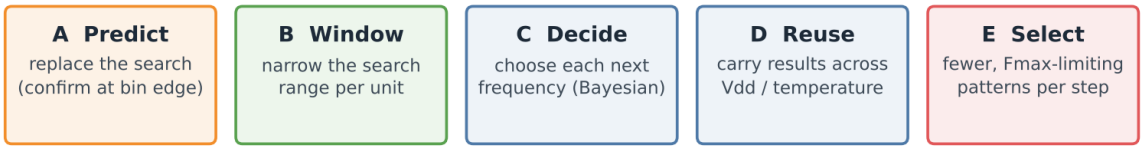


Figure 1. Five places where learning reduces Fmax search time.

3.1 A: Predictive binning (replace the search)

A regression model maps features measured earlier in the flow to Fmax. The tester confirms the predicted bin at its lower edge, with one test, and optionally checks the next bin up. The concept was demonstrated with on-chip ring-oscillator counts mapped to chip speed: running the learned model in ATE software removed the functional test from binning, and embedding the model on chip removed the ATE step as well, giving roughly three and four orders of magnitude test-time reduction respectively on a 28 nm mobile SoC, at the cost of some parts placed in a slower bin than they could have achieved [1].

This is the most aggressive option. Its weakness is that every model error becomes a binning error unless it is absorbed by a guard band, and Section 7.4 shows how large that guard band

must be unless the model error is small relative to the bin width.

3.2 B: Prediction-windowed search (narrow the search)

The model's output is not used as the answer but as a per-unit search window [L, U]. The tester runs a binary search inside the window and only tests outside it if the search runs into an edge. The result is still a measured Fmax, so the binning rule and guard band are unchanged; the model only decides where to look. A poor prediction costs extra test steps, not quality. This is the recommended starting point for most products.

3.3 C: Probabilistic (Bayesian) adaptive search

Instead of a fixed rule, the search keeps a probability distribution over the unit's Fmax and chooses each next test frequency to be most informative, typically the posterior median (probabilistic bisection) [9, 10]. After each outcome the distribution is updated with a likelihood that models test noise:

$$P(pass \mid f, F_{max}) = \Phi((F_{max} - f) / \sigma_{test})$$

The prior can be the population distribution (no ML) or the model's per-unit prediction. Because noisy outcomes only shift the distribution rather than discard half of the window, this approach is more robust to test noise than binary search, and it gives a natural stopping rule (stop when the credible interval is narrow enough) and a natural conservative estimate (a low percentile of the posterior).

3.4 D: Reuse across corners

Fmax at successive voltage or temperature points is strongly correlated. An adaptive method can use the result at one corner, together with a model of the Vdd–Fmax curve, to predict a narrow window for the next search. This idea of adaptively shrinking the frequency search space using previous measurements was shown to reduce characterisation and test cost for multi-Vdd searches [5]. In DVFS binning, an incremental model-guided Fmax search reduced system-level test applications by 45.1% against median linear search and 52.6% against binary search on 249 industrial SoCs [2].

3.5 E: Pattern selection and ordering

At each frequency step the full pattern set or workload is usually applied. Data from characterisation usually shows that a small subset of patterns or paths limits Fmax for most parts. Ordering patterns so that the most frequently failing ones run first, and stopping at the first failure, shortens failing steps; a learned subset can be used for intermediate steps, with the full set applied only at the final confirmation frequency. This reduces t_{apply} and multiplies with the step reductions from A–D.

3.6 F: Label-efficient learning

The bottleneck in building these models is that Fmax labels are expensive: each label requires a full conventional search. Semi-supervised learning uses the abundant unlabelled monitor data to learn feature representations, and was reported to reduce the number of devices that must be characterised to build a model by about six times relative to baselines [3]. Transfer learning re-uses a model trained on one product for a related product with few new labels [4]. Active learning, which chooses which units to label by model uncertainty, fits naturally with the audit-sampling loop of Section 8.

4 Data and features

4.1 Feature sources

Table 2. Typical features for Fmax prediction.

Source / insertion	Feature examples	Why it predicts Fmax
On-chip ring oscillators / speed monitors (CP, FT)	RO counts per oscillator type and location; critical-path replica delay	Directly sense local transistor speed; strongest single predictor in most studies [1, 3, 4]
Leakage (CP)	IDDQ, standby current per domain	Fast transistors (low Vt) leak more; correlated with speed
Minimum voltage (CP, FT)	Vmin of scan or memory BIST	Slow parts need more voltage to function
Process monitors	Vt, Idsat, resistor/capacitor monitors from scribe or on-die	Device-level process parameters
Structural at-speed results	Pass/fail at test frequencies, path-delay results	Partial direct measurement of timing
Location and context	Die X/Y, radius, wafer, lot, tester, site, temperature log	Captures systematic across-wafer and tool effects

4.2 Data pipeline requirements

- **Unit-level traceability.** Features from wafer sort must be joined to final-test and SLT results by an electronic chip ID (ECID) or equivalent. Without reliable unit-level joins, per-unit prediction is not possible.
- **Availability at test time.** Inference must happen before the Fmax search starts. Either the feature data is pushed to the test cell ahead of the lot, or the features are measured within the same insertion (for example, reading the on-chip monitors at the start of final test).
- **Consistent normalisation.** RO counts vary with temperature and supply; normalise to the measurement conditions, and record those conditions.
- **Versioned data.** Every prediction should be traceable to the model version and the feature values used.

4.3 Label quality

Labels are themselves noisy measurements. Use the same search resolution and conditions as the production reference, record repeat measurements on a subset to estimate test repeatability σ_{test}, and avoid training only on parts that passed a screen, which biases the model toward the fast end of the distribution.

5 Modelling

5.1 Model families

Table 3. Model families for Fmax prediction.

Model	Strengths	Limitations	When to use
Linear / ridge / PLS	Fast, interpretable, easy to embed as coefficients in a test program; PLS handles many correlated ROs	Misses non-linearity and interactions	First model; baseline; on-chip implementation
Bayesian linear regression	Predictive variance per unit; works with few labels	Linear form	Small label sets; DVFS binning [2]
Gradient-boosted trees	Handles non-linearity and mixed features; strong accuracy on tabular data	Needs calibration for intervals; extrapolates poorly	Default production model
Gaussian process regression	Native uncertainty; smooth	Scales poorly with data size	Characterisation-scale datasets
Neural nets / auto-encoders	Learn representations from unlabelled data	Data-hungry; harder to qualify	Semi-supervised and transfer settings [3, 4]
Quantile regression	Directly predicts lower/upper bounds	One model per quantile	When a conservative bound is the main output

5.2 Calibrated uncertainty

Every AI-assisted method in this document needs an uncertainty estimate, not just a point prediction. Split conformal prediction is a simple, distribution-free way to get one [8]. Hold out a calibration set of labelled units, compute residuals $r_i = y_i - \hat{y}_i$, and take empirical quantiles of those residuals. For a new unit the interval

$$[\, L, U \,] = [\, \hat{y} + q_{\alpha/2}(r),\ \hat{y} + q_{1-\alpha/2}(r) \,]$$

covers the true Fmax with probability at least $1 - \alpha$, provided the new unit comes from the same distribution as the calibration set. That proviso is why drift monitoring and out-of-distribution checks (Section 8) are mandatory. Where error varies across the population (for example, larger at the fast tail), conditional methods such as quantile regression or locally weighted conformal intervals give tighter windows.

5.3 Evaluation metrics

Table 4. Metrics to qualify an AI-optimized Fmax flow.

Metric	Definition	Target
RMSE / NRMSE	Prediction error, absolute and normalised to the Fmax range	As low as possible; about 2% NRMSE has been reported on MCU data [3, 4]
Interval coverage	Fraction of units whose true Fmax lies in [L, U]	Matches nominal level (e.g. 95%)
Mean / p95 test steps	Search steps per unit under the chosen algorithm	Lower than reference flow
Over-bin escape rate	Units placed in a faster bin than their true Fmax supports (ppm)	At or below reference flow
Under-bin rate	Units placed below their achievable bin (%)	Small; this is lost revenue
Fallback rate	Units routed to the full search	Low but non-zero

Model accuracy metrics alone are not sufficient. A model is qualified by the binning outcome it produces under the actual search algorithm and guard band, measured against the reference flow on the same units.

6 Algorithms

The algorithms below use a frequency grid G with step Δf, a test function `pass(f)` that applies the pattern set at frequency f, and a model that returns a prediction $\hat{y}$ and calibrated bounds $[L, U]$.

Algorithm 1 Prediction-windowed binary search (recommended default)

```
input : unit features x, model, grid G, interval level k
if ood(x): return full_binary_search(G)          # out-of-distribution -> reference flow
(y_hat, L, U) = model.predict_interval(x, k)
lo, hi = index(G, L), index(G, U)
best = binary_search(G[lo..hi])                  # highest passing grid point in window
if best == G[hi]:                                # hit upper edge: extend upward
    stride = 1
    while pass(G[hi + stride]): hi += stride; stride *= 2
    best = binary_search(G[hi .. hi + stride - 1])
elif best is None:                               # everything failed: search below window
    best = binary_search(G[0 .. lo - 1])
log(unit, y_hat, L, U, best, n_tests, model_version)
return best                                      # measured Fmax; bin with normal guard band
```

Algorithm 2 Probabilistic bisection with a learned prior

```
input : prior mean mu (model prediction or population mean), prior sigma s, sigma_test
p(F) = Normal(mu, s) on a fine grid
repeat until width(p, 5%..95%) <= target or n == n_max:
    f = snap_to_grid(median(p))
    outcome = pass(f)
    like(F) = Phi((F - f) / sigma_test)
    p(F) = p(F) * (like(F) if outcome else 1 - like(F)); normalise p
return quantile(p, 0.05)                         # conservative Fmax estimate
```

Algorithm 3 Confirm-only binning

```
input : prediction y_hat, sigma, bins B[0..n-1] (ascending), measurement guard band gb
b = highest bin with B[b] <= y_hat - k*sigma
if pass(B[b] + gb):
    while b+1 < n and y_hat + 2*sigma > B[b+1] and pass(B[b+1] + gb): b += 1
else:
    while b > 0:
        b -= 1
        if pass(B[b] + gb): return bin b
    return below_lowest_bin                      # reject or route to full search
return bin b                                     # 1-2 tests for most units
```

Algorithm 4 Cross-corner window reuse (DVFS)

```
for each voltage corner v_j in ascending order:
    if j == 0: window = model.predict_interval(x, v_0)
    else:      window = vdd_curve.predict(Fmax[v_0..v_(j-1)], v_j) +/- residual band
    Fmax[v_j] = windowed_binary_search(window)       # Algorithm 1 inside the window
```

7 Simulation study

7.1 Set-up

To compare strategies under identical conditions, a synthetic population was generated in which a latent process speed drives both the true Fmax and ten noisy ring-oscillator readings, together with leakage, Vmin and die position features. A portion of Fmax variation is local and invisible to the monitors, which sets a floor on achievable prediction error, as in real silicon. A gradient-boosted regression model was trained on 1,500 labelled units, calibrated on 500 and evaluated on held-out units. All searches share the same test model.

Table 5. Simulation parameters (synthetic data).

Parameter	Value
Fmax population	mean ≈ 2,000 MHz, spread ≈ 110 MHz (1 σ of the latent speed term)
Search window / grid	1,600–2,400 MHz, Δf = 10 MHz (81 grid points)
Test repeatability	σ_{test} = 4 MHz Gaussian noise on each pass/fail boundary
Time per step	25 ms pattern application + 5 ms clock switch (illustrative)
Bins	1,800 / 1,900 / 2,000 / 2,100 / 2,200 MHz; 8 MHz measurement guard band
Model	Gradient-boosted trees; held-out RMSE 26.9 MHz (≈1.3% of Fmax)
Window	$[\hat{y} - 1.5\sigma, \hat{y} + 1.5\sigma]$ with σ from the calibration residuals
Evaluation	3,000 test units per method; 60,000 units for the guard-band study

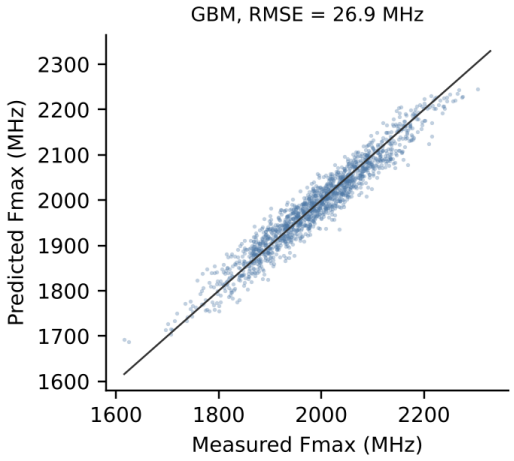


Figure 2. Predicted versus measured Fmax for held-out units. Scatter around the diagonal reflects local variation that the monitors cannot see.

7.2 Search cost

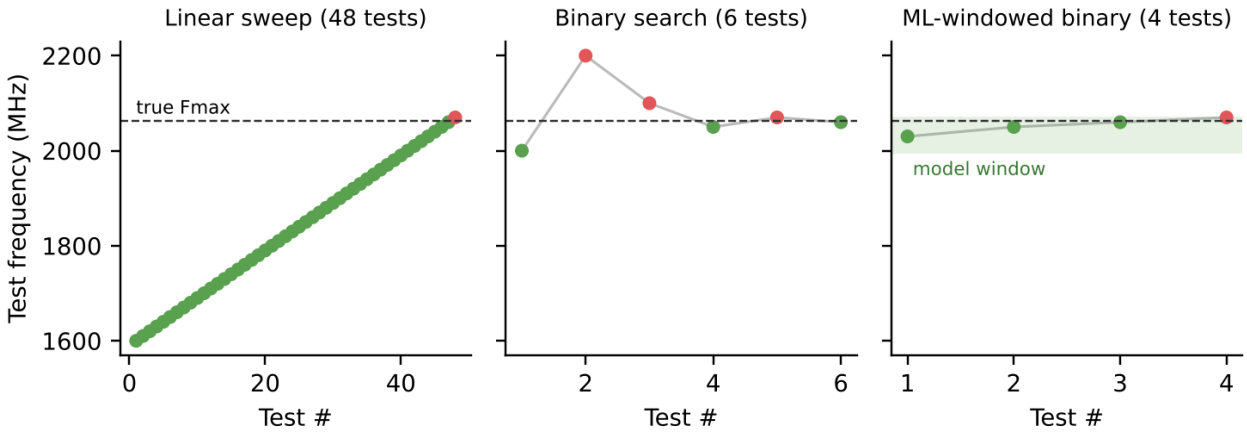


Figure 3. Example search trajectories for one unit with true Fmax near 2,063 MHz. Green points pass, red points fail. The windowed search starts inside the model's interval (shaded).

Table 6. Simulated search cost and accuracy per unit (one corner). Negative saving means slower than binary search.

Method	Mean tests	p95 tests	Time (ms)	Saving vs binary	Saving vs linear	|error| (MHz)	Under-bin
Linear sweep	40.49	58	1215	−527%	0%	5.6	9.3%
Binary search	6.45	7	194	+0%	84%	5.7	9.5%
Bayesian (population prior)	6.52	9	195	−1%	84%	7.9	11.7%
ML-seeded walk	6.17	10	185	+4%	85%	5.8	9.5%
ML-windowed binary	3.89	8	117	+40%	90%	6.0	10.0%
ML + Bayesian	4.35	7	131	+33%	89%	11.4	15.4%
ML confirm-bin (binning only)	1.77	2	53	+73%	96%	—	7.5%

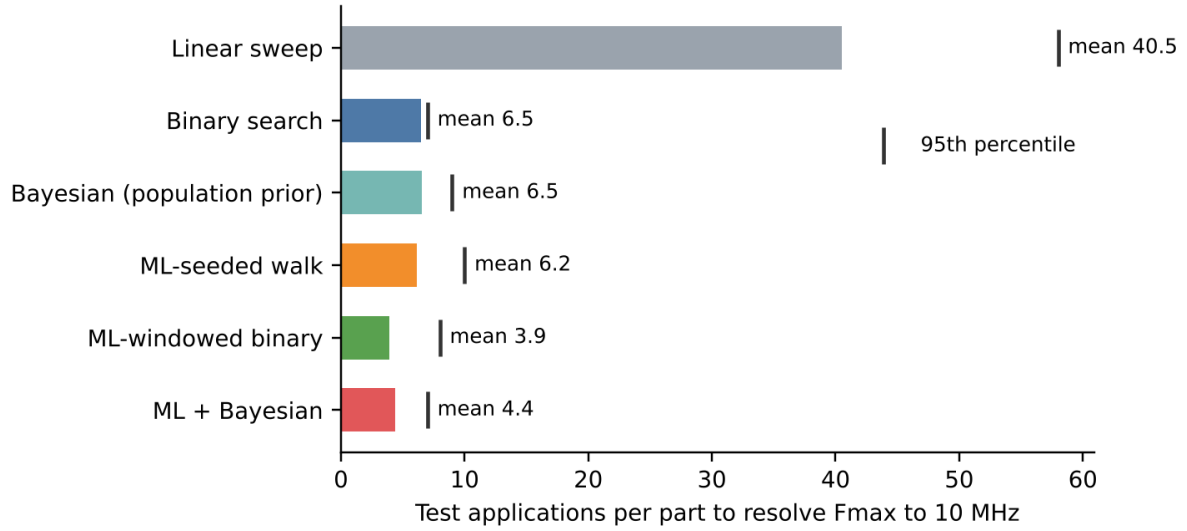


Figure 4. Mean and 95th-percentile number of test applications per unit for each strategy.

Three findings stand out. First, the prediction-windowed binary search reduced the mean number of test applications from 6.5 to 3.9, a 40% reduction relative to binary search and 90% relative to a linear sweep, while still returning a measured Fmax with the same accuracy and no over-bin escapes in the sample. This magnitude is consistent with the roughly 45–53% reduction reported for model-guided search in DVFS binning [2].

Second, the naive ML-seeded walk, which starts just below the prediction and steps up one grid point at a time, needed 6.2 tests on average, barely better than binary search and with a worse tail (p95 of 10). With a model error of about 2.7 grid steps, a walk that starts 1.5σ below the prediction spends most of its steps covering that distance. Using the prediction to bound a binary search, rather than as a starting point, is what delivers the saving.

Third, confirm-only binning needed 1.77 tests per unit. It does not measure Fmax, so it only applies where the bin is the sole output. One escape occurred among 3,000 units in this run, which is too few to estimate a ppm rate reliably; qualifying this mode needs much larger samples.

The Bayesian searches stopped once the 90% credible interval was 15 MHz wide and reported its 5th percentile, so they trade some test steps and a conservative bias (mean error -11 MHz, hence the higher under-bin rate) for robustness to noisy outcomes. Their value grows when test repeatability is poor relative to the grid step. The under-bin rate of about 10% shared by all measured methods comes from the 8 MHz measurement guard band and grid quantisation, not from the model.

7.3 Sensitivity to model accuracy

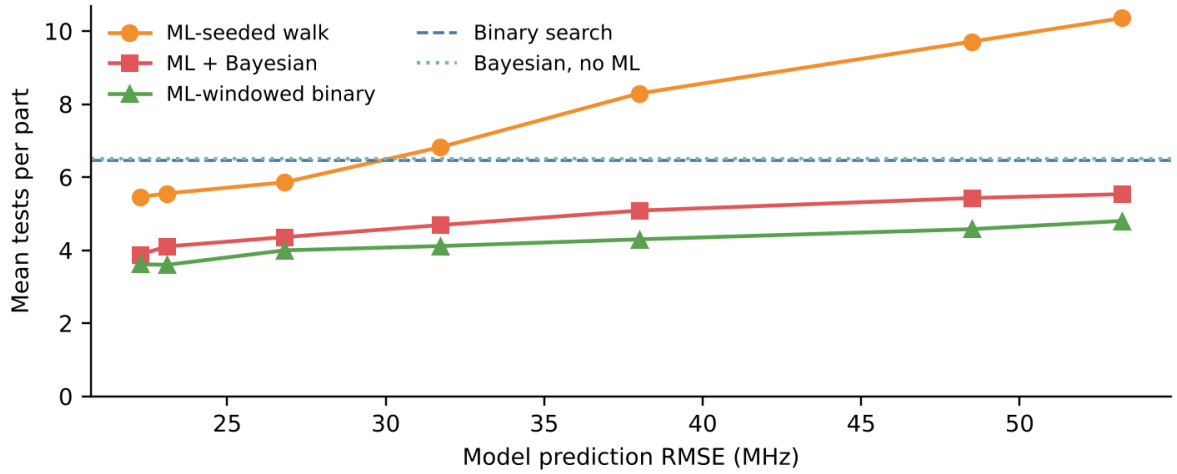


Figure 5. Mean tests per unit as the model's prediction error grows (feature noise varied). Horizontal lines show the methods that do not use a model.

The windowed search degrades gracefully: even at an RMSE of about 50 MHz, roughly half of the population spread, it still needs fewer tests than binary search because the window widens with the calibrated σ and the search stays logarithmic inside it. The naive seeded walk crosses above binary search at about 30 MHz RMSE. This is the practical argument for windowing: it fails safe with respect to both quality and test time.

7.4 Prediction-only binning and guard bands

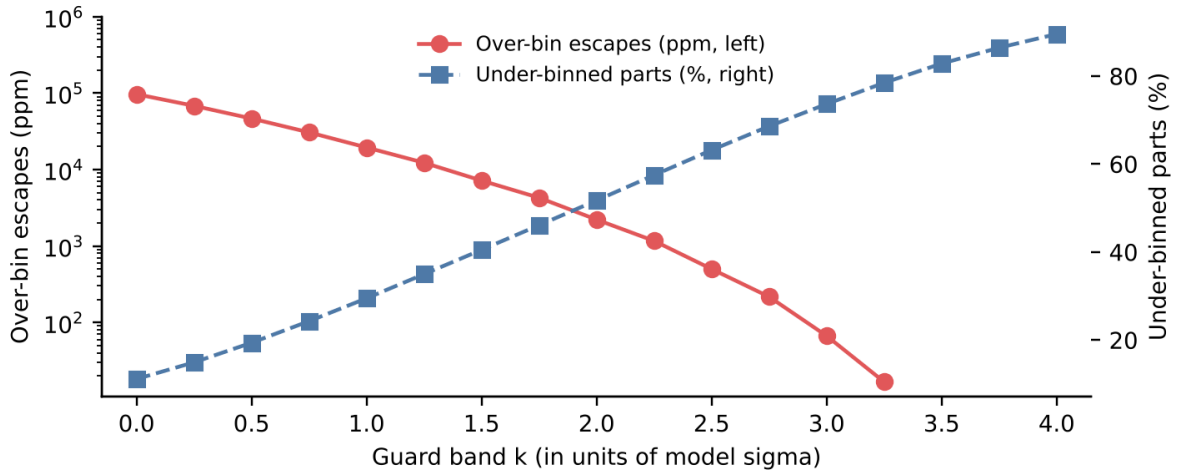


Figure 6. Prediction-only binning on 60,000 synthetic units: over-bin escapes versus under-binned parts as the guard band k (in units of model σ) increases. Points with zero observed escapes are not plotted.

If the bin is assigned from the prediction alone, the guard band must absorb the model error. With no guard band, 9.7% of units were over-binned. Reaching about 67 ppm needed k = 3, at which point 74% of units were placed below their achievable bin. Prediction-only binning is therefore viable only when model error is small compared with the bin width, as with coarse grades, very strong on-chip monitors [1], or products where under-binning is cheap. Otherwise use the windowed search, or confirm-only binning with a test at the bin edge.

7.5 Limitations of the study

- The data is synthetic; the relationships between monitors and Fmax, the noise levels and the time constants are assumptions chosen to be plausible, not measurements.
- A single corner is simulated; multi-corner reuse (Algorithm 4) would add further savings.
- Escape rates at the ppm level need far more units than simulated here for the search methods; the zero escapes observed bound the rate only to roughly 1 in 1,000.
- The simulation script is available on request and can be re-run with product-specific parameters.

8 Quality and risk management

8.1 Failure modes

Table 7. Failure modes and mitigations.

Failure mode	Effect	Mitigation
Model over-predicts Fmax	Windowed search: extra steps only. Predictive binning: over-bin escape	Measured result for binning; guard band on any predicted decision
Model under-predicts Fmax	Windowed search: edge hit, upward extension. Predictive binning: lost revenue	Edge-extension rule; monitor under-bin rate
Out-of-distribution unit (new lot, excursion, tester issue)	Interval no longer valid	OOD check on features; route to full search
Process or tester drift	Bias grows slowly; coverage drops	Audit sampling, EWMA/CUSUM on residuals, retraining
Feature data missing or late	No prediction available	Default to reference flow; alarm
Test noise near boundary	Wrong outcome flips search direction	Measurement guard band; Bayesian search; re-test at the final boundary

8.2 Out-of-distribution fallback

Before trusting a prediction, check that the unit's features lie within the training distribution: range checks on each feature, a multivariate distance such as Mahalanobis distance on the monitor vector, and lot-level statistics compared with the training data. Units that fail any check are routed to the full reference search. A non-zero fallback rate is healthy; a sudden rise is an early warning of a process or data change.

8.3 Audit sampling

A random fraction of units, typically a few percent, should receive the full conventional search regardless of the prediction. This provides unbiased labels for monitoring and retraining, and an ongoing comparison with the reference flow. To detect a bias shift δ in prediction residuals with residual standard deviation σ_r, the audit sample per monitoring period should be at least approximately

$$n \approx \left((z_{\alpha} + z_{\beta}) \cdot \sigma_r / \delta \right)^2$$

For example, detecting a 10 MHz shift with σ_r = 27 MHz at 95% confidence and 90% power needs about 60 audited units per period.

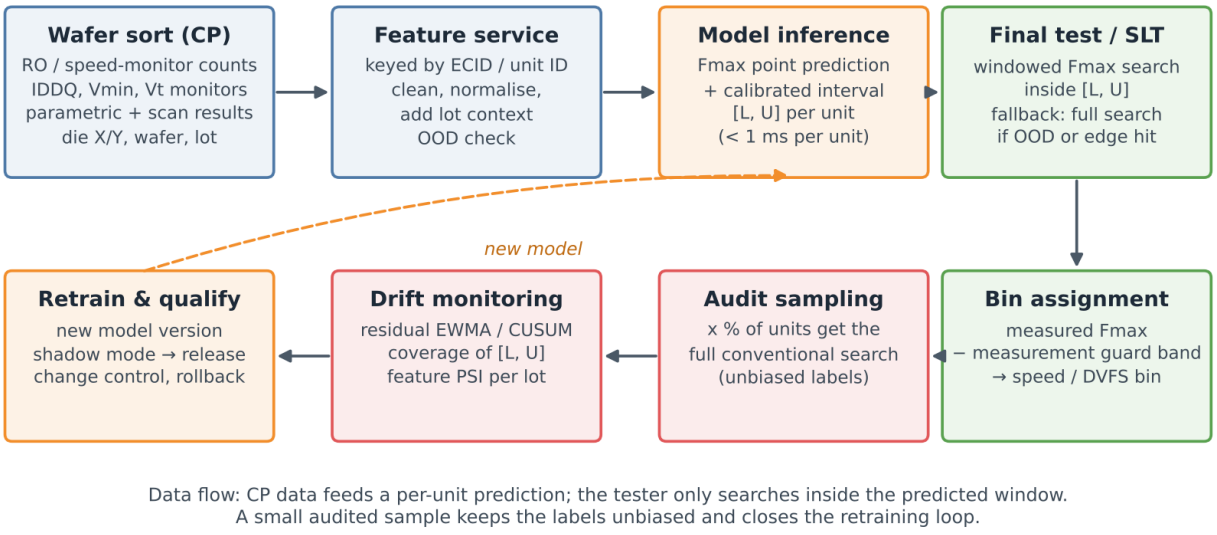
8.4 Drift monitoring

- EWMA or CUSUM control charts on the residuals of audited units, by product, tester and lot.
- Interval coverage on audited units compared with the nominal level.
- Population stability index (PSI) or similar on each input feature, per lot.
- Fallback, edge-hit and mean-test-step rates as operational indicators.

8.5 Product categories with strict quality requirements

Automotive, industrial and other high-reliability products typically require that the shipped speed grade be backed by a measurement. For these, use AI only to window or order the search (Approaches B to E) and keep the final binning decision on a measured value with the qualified guard band. Document the model and its controls in the product's test-change process, as for any other test-program change.

9 Deployment architecture



Data flow: CP data feeds a per-unit prediction; the tester only searches inside the predicted window.
A small audited sample keeps the labels unbiased and closes the retraining loop.

Figure 7. Production flow for AI-optimized Fmax search with the monitoring and retraining loop.

9.1 Integration with the tester

- **Where inference runs.** Options are (a) coefficients embedded directly in the test program, suitable for linear models; (b) a model served on the tester host or test-cell controller; or (c) predictions computed upstream after wafer sort and delivered with the lot. Option (c) removes inference from the test-time path but requires reliable data delivery and unit identification.
- **Latency budget.** Inference must add negligible time; well under 1 ms per unit is achievable for tree ensembles and linear models.
- **Portable model format.** Export models in a portable form (for example ONNX or a coefficient table) with a version identifier stored in the test log.
- **On-chip binning.** When the model is simple enough, it can be evaluated on chip from its own monitors, as in built-in self-speed-binning [1].

9.2 Model lifecycle

Table 8. Model lifecycle stages.

Stage	Activity	Exit criterion
Characterisation	Full search on a representative sample across lots and corners	Labels cover the process window
Shadow mode	Model predicts in production; decisions still from reference flow	Coverage, test-step and binning metrics meet targets on production data
Limited release	AI flow on a fraction of lots, side-by-side comparison	No escape increase; savings confirmed
Full release	AI flow default with audit sampling and drift monitoring	Ongoing control limits
Retrain	Triggered by drift alarms or scheduled; re-qualified in shadow mode	As for shadow mode
Rollback	One-switch return to the reference flow	Always available

9.3 Estimating the benefit

The annual saving from reduced search time is

$$Saving = N_{units} \times N_{corners} \times \Delta N_{steps} \times (t_{switch} + t_{apply}) \times C_{test}$$

where C_{test} is the cost per unit-second of tester time. As an illustration only: an SLT Fmax search at three voltage corners, with 0.4 s per step and a reduction of 2.6 steps per corner (binary to windowed, from Section 7), saves about 3.1 s per unit. At 20 million units a year and an assumed cost of $0.004 per unit-second, that is about $246k a year, before counting the tester capacity released. Substitute product-specific values for every term.

10 Reported results

Table 9. Selected published results (paraphrased).

Work	Approach	Data	Reported outcome
Mu, Chao et al. [1]	RO counts → chip-speed model; in ATE software and as on-chip built-in self-speed-binning	360 chips, 28 nm, 0.9 V, 1.6 GHz mobile SoC	About 10^3× (ATE) and 10^4× (on-chip) test-time reduction vs functional binning; some parts binned lower
Chang & Chao [2]	Bayesian linear regression with stepwise feature selection for DVFS bins; incremental model-guided Fmax search	249 industrial SoCs	Accurate DVFS binning with no slow part placed in a faster bin; 45.1% / 52.6% fewer SLT applications vs median linear / binary search
Bellarmino et al. [3]	Semi-supervised deep feature extraction + linear regression	Industrial MCU speed-monitor data	About 6× fewer characterised devices needed to build the model; ~2% NRMSE
Bellarmino et al. [4]	Inter-product transfer learning	MCU products	~2% NRMSE with reduced labelling
DATE 2013 [5]	Adaptive multi-level reduction of the Fmax search space across Vdd levels	Benchmark circuits	Reduced characterisation / test time for multi-Vdd Fmax search
Sensors 2022 [6]	On-chip binning checkers and adaptors for Fmax yield optimisation	ITC, ISCAS89, OpenSPARC T2 on 28 nm FPGAs	Promotes parts to higher bins with negligible area and test overhead

Related tools address other parts of the test-cost problem. AI-driven ATPG optimisation, such as Synopsys TSO.ai, tunes pattern-generation settings to reduce pattern count and test cycles [7]; this shortens t_{apply} for structural tests and is complementary to Fmax search optimisation.

11 Implementation roadmap

Table 10. Phased implementation plan.

Phase	Duration (typical)	Deliverables
0 Feasibility	2–4 weeks	Data audit: unit traceability, monitor availability at test time, existing Fmax logs; baseline test-time breakdown by corner
1 Offline model	4–6 weeks	Feature pipeline; baseline linear model and GBM; conformal intervals; offline replay of Algorithms 1 and 3 on historical data
2 Shadow mode	1–2 months	Inference in the test cell; logging; coverage and test-step metrics on live lots; OOD rules
3 Limited release	1–2 months	Windowed search on selected lots with audit sampling; escape and under-bin comparison with reference flow
4 Full release	ongoing	Default flow; drift dashboards; retraining procedure; extension to multi-corner reuse and pattern ordering

12 Conclusions

AI-optimized Fmax search reduces test time by using data the flow already collects to tell the tester where to look. The most dependable form keeps the final decision on a measured value and uses a calibrated per-unit prediction only to bound the search, giving large reductions in search steps with no change to the binning rule. More aggressive forms, such as confirm-only or prediction-only binning, deliver more savings but need accurate models, wide guard bands or strong on-chip monitors, and much larger qualification samples. In every form, production safety depends on the surrounding controls: out-of-distribution fallback, audit sampling, drift monitoring and a disciplined model lifecycle.

References

[1] S.-P. Mu, M. C.-T. Chao, S.-H. Chen, Y.-M. Wang, “Statistical framework and built-in self-speed-binning system for speed binning using on-chip ring oscillators,” *IEEE Trans. VLSI Systems*, 2016. https://ir.lib.nycu.edu.tw/handle/11536/133641

[2] K.-W. Chang, M. C.-T. Chao, “DVFS binning using machine-learning techniques,” M.S. thesis, National Chiao Tung University, 2017. https://ir.lib.nycu.edu.tw/handle/11536/142720

[3] N. Bellarmino et al., “Semi-supervised deep learning for microcontroller performance screening,” *Proc. IEEE European Test Symposium (ETS)*, 2023. https://iris.polito.it/retrieve/handle/11583/2976248/623936

[4] N. Bellarmino et al., “Enabling inter-product transfer learning on MCU performance screening,” *Proc. IEEE Asian Test Symposium (ATS)*, 2023. https://iris.polito.it/retrieve/handle/11583/2981872/667277

[5] “Adaptive reduction of the frequency search space ...,” *Proc. Design, Automation and Test in Europe (DATE)*, 2013, interactive presentation IP1-19. https://past.date-conference.com/proceedings-archive/2017/pyear/PAPERS/2013/DATE13/PDFFILES/IP1_19.PDF

[6] “On-chip structures for Fmax binning and optimization,” *Sensors*, vol. 22, no. 4, art. 1382, 2022. https://www.mdpi.com/1424-8220/22/4/1382

[7] Synopsys, “TSO.ai datasheet,” 2023. https://www.synopsys.com/resources/ai-powered-eda-tsoai-ds.html

[8] A. N. Angelopoulos, S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” arXiv:2107.07511, 2021.

[9] M. Horstein, “Sequential transmission using noiseless feedback,” *IEEE Trans. Information Theory*, vol. 9, no. 3, 1963.

[10] R. Waeber, P. I. Frazier, S. G. Henderson, “Bisection search with noisy responses,” *SIAM J. Control and Optimization*, vol. 51, no. 3, 2013.

[11] B. D. Cory, R. Kapur, B. Underwood, “Speed binning with path delay test in 150-nm technology,” *IEEE Design & Test of Computers*, vol. 20, no. 5, 2003.

[12] J. Zeng et al., “On correlating structural tests with functional tests for speed binning of high performance design,” *Proc. IEEE International Test Conference (ITC)*, 2004.

[13] Wide-range clock signal generation for speed grading of logic cores, U.S. Patent 10,317,462. https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/10317462

Appendix A Glossary

Table 11. Glossary.

Term	Meaning
ATE	Automatic test equipment
CP / FT / SLT	Chip probe (wafer sort) / final test / system-level test
DVFS	Dynamic voltage and frequency scaling; bins define voltage–frequency tables
ECID	Electronic chip identifier used for unit-level traceability
Fmax	Maximum frequency at which a unit passes the defined test at given conditions
Guard band	Margin applied to a measured or predicted value before a binning decision
NRMSE	Root-mean-square error normalised to the range of the target
OOD	Out of distribution: inputs unlike the training data
Over-bin escape	Unit placed in a faster bin than its true Fmax supports
RO / SMON	Ring oscillator / speed monitor: on-chip circuits whose frequency tracks transistor speed
Shmoo	Pass/fail map over two test conditions, e.g. frequency and voltage